Exercício sobre Construtores em Java

Instruções

Crie uma classe `Carro` que possua três atributos: `marcaDoCarro`, `modeloDoCarro` e `anoDoCarro`. Implemente três construtores diferentes para a classe:

1. Um construtor padrão que não recebe parâmetros.
2. Um construtor que recebe `marca` e `modelo`.
3. Um construtor que recebe `marca`, `modelo` e `ano`.

Passos para Realizar o Exercício

1. Crie uma nova classe Java chamada `Carro`.
2. Declare os atributos `marcaDoCarro`, `modeloDoCarro` e `anoDoCarro` na classe `Carro`.
3. Implemente o construtor padrão que inicializa os atributos com valores padrão ("Desconhecida", "Desconhecido", 0).
4. Implemente o construtor que recebe `marca` e `modelo` e inicializa os atributos correspondentes. O `anoDoCarro` deve receber o valor padrão 0.
5. Implemente o construtor que recebe `marca`, `modelo` e `ano` e inicializa todos os atributos.
6. Crie um método `exibirDetalhes()` que imprime as informações do carro e pula uma linha para melhor organização visual (`System.out.println()`).
7. Crie uma classe `Aplicacao` com o método `main` para testar os diferentes construtores.
8. No método `main`, crie três objetos `Carro` usando cada um dos construtores e chame o método `exibirDetalhes()` para cada um.

Observações Importantes

- **Nomes dos atributos**: Use `marcaDoCarro`, `modeloDoCarro` e `anoDoCarro` para facilitar a distinção entre os atributos da classe e os parâmetros dos construtores.
- **Execução**: Lembre-se de executar a classe `Aplicacao`, pois é ela que contém o método `main`. Se você executar a classe `Carro`, receberá um erro dizendo que a classe `Carro` não contém o método `main`: **"can't find main(String[]) method in class: Carro"**.
- **Saída organizada**: O método `exibirDetalhes()` deve incluir uma linha em branco no final (`System.out.println()`) para separar visualmente as informações de cada carro.

Código do exercício

```
class Carro {
    String marcaDoCarro;
    String modeloDoCarro;
    int anoDoCarro;
    // Construtor padrão
    public Carro() {
        marcaDoCarro = "Desconhecida";
        modeloDoCarro = "Desconhecido";
        anoDoCarro = 0;
    }
    // Construtor com dois parâmetros
    public Carro(String marca, String modelo) {
        marcaDoCarro = marca;
        modeloDoCarro = modelo;
        anoDoCarro = 0; // Valor padrão para ano
    }
    // Construtor com três parâmetros
    public Carro(String marca, String modelo, int ano) {
        marcaDoCarro = marca;
        modeloDoCarro = modelo;
        anoDoCarro = ano;
    }
    // Método para exibir os detalhes do carro
    public void exibirDetalhes() {
        System.out.println("Marca: " + marcaDoCarro);
        System.out.println("Modelo: " + modeloDoCarro);
        System.out.println("Ano: " + anoDoCarro);
        System.out.println(); // Pula uma linha para mostrar o próximo
carro
    }
}
```

```
public class Aplicacao {
    public static void main(String[] args) {
        // Testando o construtor padrão
        Carro carro1 = new Carro();
        carro1.exibirDetalhes();
        // Testando o construtor com dois parâmetros
        Carro carro2 = new Carro("Toyota", "Corolla");
        carro2.exibirDetalhes();
        // Testando o construtor com três parâmetros
        Carro carro3 = new Carro("Honda", "Civic", 2020);
        carro3.exibirDetalhes();
    }
}
```

Explicação do Código

1. **Construtor Padrão**: Inicializa os atributos `marca`, `modelo` e `ano` com valores padrão.
2. **Construtor com Dois Parâmetros**: Inicializa `marca` e `modelo` com os valores fornecidos e `ano` com um valor padrão.
3. **Construtor com Três Parâmetros**: Inicializa todos os atributos com os valores fornecidos.
4. **Método `exibirDetalhes`**: Exibe os detalhes do carro no console.
5. **Classe `Aplicacao`**: Testa os diferentes construtores criando instâncias da classe `Carro` e exibindo seus detalhes.

Exercício: Usando Métodos e Encapsulamento (private)

Instruções

Este exercício tem como objetivo testar seu entendimento sobre encapsulamento em Java, utilizando os modificadores de acesso `public` e `private`. Você criará uma classe `Produto` com atributos que representam as características de um produto, utilizando apenas construtores para inicializar os atributos.

Passos para fazer o exercício

1. Crie uma classe chamada `Produto`.
2. Adicione três atributos à classe `Produto`: `nome`, `preco` e `quantidadeEmEstoque`.
3. Utilize o modificador de acesso `private` para os atributos.
4. Crie um construtor público para inicializar os atributos.
5. Teste a classe `Produto` em uma classe principal, criando instâncias e exibindo os atributos através de um método de exibição.

Código do exercício

```java
// Classe Produto
public class Produto {
    // Atributos privados
    private String nome;
    private double preco;
    private int quantidadeEmEstoque;

    // Construtor público
    public Produto(String nome, double preco, int quantidadeEmEstoque) {
        this.nome = nome;
        this.preco = preco;
        this.quantidadeEmEstoque = quantidadeEmEstoque;
    }

    // Método para exibir informações do produto
    public void exibirInformacoes() {
        System.out.println("Nome: " + nome);
        System.out.println("Preço: " + preco);
        System.out.println("Quantidade em Estoque: " +
quantidadeEmEstoque);
    }
}
```

```java
// Classe principal para testar Produto
public class Main {
    public static void main(String[] args) {
        // Criando instâncias de Produto
        Produto produto1 = new Produto("Notebook", 2500.00, 10);
        Produto produto2 = new Produto("Smartphone", 1500.00, 20);

        // Exibindo informações dos produtos
        produto1.exibirInformacoes();
        produto2.exibirInformacoes();
    }
}
```

Explicação do código

- **Classe `Produto`**: Contém três atributos (`nome`, `preco`, `quantidadeEmEstoque`) que são privados. Isso significa que eles não podem ser acessados diretamente fora da classe.
- **Construtor**: Um construtor público é utilizado para inicializar os atributos privados quando uma nova instância da classe `Produto` é criada.
- **Método `exibirInformacoes`**: Este método público é utilizado para exibir os valores dos atributos. Ele permite que as informações do produto sejam acessadas de forma controlada.
- **Classe `Main`**: Cria instâncias da classe `Produto` e utiliza o método `exibirInformacoes` para mostrar os atributos dos produtos. Isso demonstra como o encapsulamento protege os dados e fornece uma interface segura para interagir com eles.

Exercício: Usando static em Métodos e Variáveis Membro

Instruções

Este exercício tem como objetivo ensinar o uso de variáveis e métodos `static` em Java, utilizando duas classes no mesmo arquivo. Você irá criar uma classe que utiliza variáveis e métodos estáticos para contar o número de alunos em uma sala de aula e outra classe para testar essa funcionalidade.

Passos para fazer o exercício

1. Crie uma classe chamada `SalaDeAula`.
2. Dentro da classe `SalaDeAula`, declare uma variável estática para armazenar o número total de alunos.
3. Crie um método estático para adicionar alunos à contagem.
4. Crie um método estático para exibir o número total de alunos.
5. Crie uma segunda classe chamada `TesteSalaDeAula` em um segundo arquivo.
6. No método `main` da classe `TesteSalaDeAula`, crie três instâncias da classe `SalaDeAula` e adicione alunos para testar a funcionalidade.

Obs.: Lembre-se que não pode haver duas classes `public` no mesmo arquivo, senão o programa não compilará.

Código do exercício

```java
public class SalaDeAula {
    // Variável estática para contar o número total de alunos
    static int totalAlunos = 0;

    // Método estático para adicionar alunos
    public static void adicionarAluno() {
        totalAlunos++;
    }

    // Método estático para exibir o número total de alunos
    public static void exibirTotalAlunos() {
        System.out.println("Total de alunos na sala: " + totalAlunos);
    }
}
```

```java
public class TesteSalaDeAula {
    public static void main(String[] args) {
        // Criando instâncias da classe SalaDeAula
        SalaDeAula sala1 = new SalaDeAula();
        SalaDeAula sala2 = new SalaDeAula();
        SalaDeAula sala3 = new SalaDeAula();

        // Adicionando alunos
        SalaDeAula.adicionarAluno();
        SalaDeAula.adicionarAluno();
        SalaDeAula.adicionarAluno();

        // Exibindo o número total de alunos
        SalaDeAula.exibirTotalAlunos();
    }
}
```

Explicação do código

1. **Classe `SalaDeAula`**:
    - **Variável estática `totalAlunos`**: Esta variável é usada para contar o número total de alunos na sala de aula. Como é estática, ela pertence à classe e não a instâncias individuais.
    - **Método estático `adicionarAluno`**: Este método incrementa a variável `totalAlunos` em 1 cada vez que é chamado.
    - **Método estático `exibirTotalAlunos`**: Este método exibe o valor atual da variável `totalAlunos`.
2. **Classe `TesteSalaDeAula`**:
    - **Método `main`**: Este método cria três instâncias da classe `SalaDeAula` para ilustrar que a variável `totalAlunos` é compartilhada entre todas as instâncias. Em seguida, chama o método `adicionarAluno` três vezes para simular a adição de alunos e, finalmente, chama o método `exibirTotalAlunos` para mostrar o total de alunos na sala.

Exercício Complementar: Construtores em Java

Instruções

Crie uma classe `Livro` com os seguintes atributos: `titulo`, `autor`, `anoPublicacao` e `preco`. Utilize diferentes modificadores de acesso para cada atributo e implemente três construtores diferentes para a classe:

1. Um construtor "padrão" que não recebe parâmetros.
2. Um construtor que recebe `titulo` e `autor`.
3. Um construtor que recebe `titulo`, `autor`, `anoPublicacao` e `preco`.

Passos para Realizar o Exercício

1. Crie uma nova classe Java chamada `Livro`.

2. Declare os atributos `titulo`, `autor`, `anoPublicacao` e `preco` na classe `Livro` com os seguintes modificadores de acesso:

   - `titulo`: `public`
   - `autor`: `private`
   - `anoPublicacao`: `protected`
   - `preco`: `default` (sem modificador)

3. Implemente os três construtores na classe `Livro`. Esses construtores permitem criar objetos Livro com diferentes níveis de detalhe, dependendo das informações disponíveis no momento da criação.

   - Construtor "padrão": Inicializa os atributos `titulo`, `autor`, `anoPublicacao` e `preco` com valores padrão (“Desconhecido”, “Desconhecido”, 0, 0.0).
   - Construtor com dois parâmetros: Permite definir os atributos `titulo` e `autor`, enquanto `anoPublicacao` e `preco` são inicializados com valores padrão (0 e 0.0).
   - Construtor com quatro parâmetros: Permite definir todos os atributos (`titulo`, `autor`, `anoPublicacao` e `preco`) com valores fornecidos.

4. Crie uma classe `Aplicacao` com o método `main` para testar os diferentes construtores e acessar diretamente as variáveis membro.

Código do Exercício

```
class Livro {
    public String titulo;
    private String autor;
    protected int anoPublicacao;
    double preco; // Modificador default

    // Construtor padrão
    public Livro() {
        this.titulo = "Desconhecido";
        this.autor = "Desconhecido";
        this.anoPublicacao = 0;
        this.preco = 0.0;
    }

    // Construtor com dois parâmetros
    public Livro(String titulo, String autor) {
        this.titulo = titulo;
        this.autor = autor;
        this.anoPublicacao = 0; // Valor padrão para anoPublicacao
        this.preco = 0.0; // Valor padrão para preco
    }

    // Construtor com quatro parâmetros
    public Livro(String titulo, String autor, int anoPublicacao, double
preco) {
        this.titulo = titulo;
        this.autor = autor;
        this.anoPublicacao = anoPublicacao;
        this.preco = preco;
    }
}
```

```
public class Aplicacao {
    public static void main(String[] args) {
        // Testando o construtor padrão
        Livro livro1 = new Livro();
        System.out.println("Título: " + livro1.titulo);
        // Não é possível acessar diretamente 'autor' pois é private
        // System.out.println("Autor: " + livro1.autor);
        System.out.println("Ano de Publicação: " + livro1.anoPublicacao);
        System.out.println("Preço: " + livro1.preco);

        // Testando o construtor com dois parâmetros
        Livro livro2 = new Livro("1984", "George Orwell");
        System.out.println("Título: " + livro2.titulo);
        // Não é possível acessar diretamente 'autor' pois é private
        // System.out.println("Autor: " + livro2.autor);
        System.out.println("Ano de Publicação: " + livro2.anoPublicacao);
```

```
        System.out.println("Preço: " + livro2.preco);

        // Testando o construtor com quatro parâmetros
        Livro livro3 = new Livro("O Senhor dos Anéis", "J.R.R. Tolkien",
1954, 49.90);
        System.out.println("Título: " + livro3.titulo);
        // Não é possível acessar diretamente 'autor' pois é private
        // System.out.println("Autor: " + livro3.autor);
        System.out.println("Ano de Publicação: " + livro3.anoPublicacao);
        System.out.println("Preço: " + livro3.preco);
    }
}
```

Explicação do Código

1. **Construtor Padrão**: Inicializa os atributos `titulo`, `autor`, `anoPublicacao` e `preco` com valores padrão.
2. **Construtor com Dois Parâmetros**: Inicializa `titulo` e `autor` com os valores fornecidos e `anoPublicacao` e `preco` com valores padrão.
3. **Construtor com Quatro Parâmetros**: Inicializa todos os atributos com os valores fornecidos.
4. **Acesso Direto**: Atributos `titulo`, `anoPublicacao` e `preco` são acessados diretamente no método `main`, enquanto `autor` não pode ser acessado diretamente por ser `private`.

Exercício Complementar: Modificadores de Acesso em Java

Instruções

Crie uma classe `Pessoa` com os seguintes atributos: `nome`, `idade`, `endereco` e `telefone`. Utilize diferentes modificadores de acesso para cada atributo e implemente métodos para acessar e modificar esses atributos.

Passos para Realizar o Exercício

1. Crie uma nova classe Java chamada `Pessoa`.
2. Declare os atributos `nome`, `idade`, `endereco` e `telefone` na classe `Pessoa` com os seguintes modificadores de acesso:
   - `nome`: `public`
   - `idade`: `private`
   - `endereco`: `protected`
   - `telefone`: `default` (sem modificador)
3. Implemente métodos `get` e `set` para os atributos `idade`, `endereco` e `telefone`. Por exemplo:

   ```java
   public int getIdade() {
       return idade;
   }

   public void setIdade(int idade) {
       this.idade = idade;
   }
   ```

4. Crie uma classe `Aplicacao` com o método `main` para testar a classe `Pessoa`.

Código do Exercício

```
class Pessoa {
    public String nome;
    private int idade;
    protected String endereco;
    String telefone; // Modificador default

    // Construtor
    public Pessoa(String nome, int idade, String endereco, String telefone)
{
        this.nome = nome;
        this.idade = idade;
        this.endereco = endereco;
        this.telefone = telefone;
    }

    // Métodos get e set para idade
    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }

    // Métodos get e set para endereco
    public String getEndereco() {
        return endereco;
    }

    public void setEndereco(String endereco) {
        this.endereco = endereco;
    }

    // Métodos get e set para telefone
    public String getTelefone() {
        return telefone;
    }

    public void setTelefone(String telefone) {
        this.telefone = telefone;
    }
}
```

```
public class Aplicacao {
    public static void main(String[] args) {
        Pessoa pessoa = new Pessoa("João", 30, "Rua A, 123", "1234-5678");

        // Acessando e modificando atributos
        System.out.println("Nome: " + pessoa.nome); // Acesso direto
(public)
        System.out.println("Idade: " + pessoa.getIdade()); // Acesso via
método (private)
        pessoa.setIdade(31);
        System.out.println("Nova Idade: " + pessoa.getIdade());

        System.out.println("Endereço: " + pessoa.getEndereco()); // Acesso
via método (protected)
        pessoa.setEndereco("Rua B, 456");
        System.out.println("Novo Endereço: " + pessoa.getEndereco());

        System.out.println("Telefone: " + pessoa.getTelefone()); // Acesso
via método (default)
        pessoa.setTelefone("8765-4321");
        System.out.println("Novo Telefone: " + pessoa.getTelefone());
    }
}
```

Explicação do Código

1. **Modificador `public`**: O atributo `nome` pode ser acessado diretamente de qualquer lugar.
2. **Modificador `private`**: O atributo `idade` só pode ser acessado e modificado através dos métodos `getIdade` e `setIdade`.
3. **Modificador `protected`**: O atributo `endereco` pode ser acessado e modificado através dos métodos `getEndereco` e `setEndereco`.
4. **Modificador `default`**: O atributo `telefone` pode ser acessado e modificado através dos métodos `getTelefone` e `setTelefone`.

Exercicio Complementar: Passagem por Valor e Passagem por Referência

Este exercício tem como objetivo demonstrar a diferença entre passagem por valor e passagem por referência em Java. Você irá criar dois métodos: um que tenta modificar um valor primitivo e outro que modifica um objeto.

Passos para fazer o exercício

1. **Crie uma classe chamada `Personagem`**:
    - Defina os atributos necessários, como `nome` e `nivelDePoder`.
    - Crie um construtor para inicializar esses atributos.
2. **Dentro dessa classe, crie um método que tenta modificar um valor primitivo**:
    - Este método deve receber um parâmetro do tipo `int` e tentar modificar seu valor.
    - Por exemplo, você pode criar um método chamado `tentarAumentarNivelDePoder` que recebe um `int` e adiciona 10 a ele.
3. **Crie outro método que modifica um objeto**:
    - Este método deve receber um objeto do tipo `Personagem` e modificar um de seus atributos.
    - Por exemplo, você pode criar um método chamado `mudarNome` que recebe um objeto `Personagem` e altera o atributo `nome` desse objeto.
4. **No método `main`, crie uma instância da classe `Personagem` e chame os métodos, observando os resultados**:
    - Crie uma instância da classe `Personagem` com valores iniciais para `nome` e `nivelDePoder`.
    - Chame o método que tenta modificar o valor primitivo e imprima o valor de `nivelDePoder` antes e depois da chamada para observar que ele não muda.
    - Chame o método que modifica o objeto e imprima o valor de `nome` antes e depois da chamada para observar que ele muda.
    - Adicionalmente, você pode criar um método que realmente modifica o `nivelDePoder` do objeto e chamar esse método para ver a mudança.

Código do exercício

```
class Personagem {
    String nome;
    int nivelDePoder;

    Personagem(String nome, int nivelDePoder) {
        this.nome = nome;
        this.nivelDePoder = nivelDePoder;
    }

    // Método que tenta modificar um valor primitivo
    public void tentarAumentarNivelDePoder(int nivelDePoder) {
        nivelDePoder += 10;
    }

    // Método que modifica um objeto
    public void mudarNome(Personagem personagem) {
        personagem.nome = "Guerreiro";
    }

    // Método que realmente aumenta o nível de poder do personagem
    public void aumentarNivelDePoder() {
        this.nivelDePoder += 10;
    }
}

public class PassagemParametrosRPG {

    public static void main(String[] args) {
        Personagem heroi = new Personagem("Mago", 50);

        System.out.println("Antes de tentarAumentarNivelDePoder: " +
heroi.nivelDePoder);
        heroi.tentarAumentarNivelDePoder(heroi.nivelDePoder);
        System.out.println("Depois de tentarAumentarNivelDePoder: " +
heroi.nivelDePoder);

        System.out.println("Antes de mudarNome: " + heroi.nome);
        heroi.mudarNome(heroi);
        System.out.println("Depois de mudarNome: " + heroi.nome);

        System.out.println("Antes de aumentarNivelDePoder: " +
heroi.nivelDePoder);
        heroi.aumentarNivelDePoder();
        System.out.println("Depois de aumentarNivelDePoder: " +
heroi.nivelDePoder);
    }
}
```

Explicação do código

1. **Passagem por Valor**: No método `tentarAumentarNivelDePoder`, passamos um valor primitivo (`int`). Quando chamamos esse método no `main`, ele recebe uma cópia do valor original. Portanto, qualquer modificação feita dentro do método não afeta a variável original. É por isso que o valor de `nivelDePoder` permanece 50 antes e depois da chamada do método.

2. **Passagem por Referência**: No método `mudarNome`, passamos um objeto `Personagem`. Em Java, os objetos são passados por referência, o que significa que o método recebe uma referência ao objeto original. Qualquer modificação feita no objeto dentro do método afeta o objeto original. É por isso que o nome do personagem muda de "Mago" para "Guerreiro" após a chamada do método.

3. **Método Adicional**: Adicionamos um método `aumentarNivelDePoder` que realmente aumenta o nível de poder do personagem. Este método modifica o atributo `nivelDePoder` diretamente, mostrando como a passagem por referência permite modificar o estado interno do objeto.